RAG evaluation and chunking comparison

Ticket Easy knowledge service (services/knowledge). Written 28 October 2026.

This report does two things:

It collects every RAG evaluation result already in the repository, and marks each metric as measured or missing.It reports a new controlled experiment that compares fixed-size, recursive, semantic and structure-aware chunking on the repository's own retrieval benchmarks.Every number in this report is either copied from a named file in the repository or produced by the scripts listed in §13. docs/reports/eval_runs/chunking/verify_report.py checks the tables in this report against those files.

Bottom line. Keep the production chunking, which is structure-aware: one passage per heading or FAQ row. None of the eight alternatives we tested beat it significantly on hit@5. It has the highest mean context recall in two of the three settings. In the third, semantic-lex-95 has a higher mean, but it returns 2.3 times as much text and does worse than structure on more individual questions (45 lost, 12 won). Under an equal context budget, structure also has the best char precision, meaning the highest share of gold text in what it returns. Two important limits apply:

The experiment ran in keyword-only (BM25) mode, because Ollama and bge-m3 were not available on this machine.Faithfulness and Answer Correctness could not be measured, because no LLM judge or generator was available.Both are listed under remaining work (§10).

1. What the repository contains

	Notes
	Structure-aware. Markdown ##+ headings, DOCX heading styles, numbered PDF headings, one XLSX row per FAQ, HTML h2–h6. A section i
	Ollama bge-m3, unit-normalised, LRU-cached
	Hybrid: 0.65·cosine + 0.35·BM25/(BM25+min_bm25). Query gate: min cosine OR min BM25; per-tenant thresholds in the manifest. Arabizi/Ara
	LLM on the Groq free tier, so it is outside the knowledge service. A deterministic check_grounded blocks replies that contain numbers, dates, id
	shop_001: 33 dev / 37 test. noon_eg: 65 / 64. telco_001: 102 / 102. Gold = section citations, where any listed citation counts. Unanswerable qu
s/reports/eval_runs/models/replay_analysis.py	Retrieval metrics: recall@5, no-answer precision/recall. The agent side scores citation accuracy and unbacked facts.
	Section hit, abstention, optional LLM contextual relevancy and faithfulness, plus 30 hand faithfulness labels. No judge scores are committed.

2. Existing evaluation results, aggregated

2.1 Measured: retrieval, hybrid bge-m3 + BM25, production chunking

Source: docs/reports/eval_runs/knowledge/summary.json and the eval-retrieval_*.txt outputs next to it, recorded at commit 0be10fd on 16 October 2026.

Tenant	Split	Questions (unanswerable)	Recall@5 %	No-answer precision %	No-answer recall %	ar	arabizi	en
shop_001	test	37 (9)	89.29	87.5	77.78	7/9	10/10	8/9
shop_001	dev	33 (6)	92.59	100.0	50.0	8/9	9/9	8/9
noon_eg	test	64 (24)	97.5	93.75	62.5	12/12	11/12	16/16
noon_eg	dev	65 (24)	90.24	90.0	75.0	11/12	11/13	15/16
telco_001	test	102 (12)	82.22	40.0	33.33	27/30	19/30	28/30
telco_001	dev	102 (12)	88.89	72.73	66.67	27/30	27/30	26/30

Other measured knowledge-service results from the same source: noon scenarios, expected-section recall@5 75.46% (246/326 sections), 45/64 no-evidence scenarios returned empty; guardrails 44/44, 0 blocked actions executed.

2.2 Measured: end-to-end figures related to RAG quality

	Source	What it is, and what it is not
	docs/reports/eval_runs/tables/headline.csv	The reply cites the expected policy passage. This is closest to answer correctness at the cita
	same file	The deterministic check finds no number, date, id or link in a sent reply that its evidence lack
d language	docs/reports/eval_runs/models/replay_comparison.csv	The same check_grounded rule applied to replayed model outputs. Agreement is measured
	docs/reports/eval_runs/usage/knowledge_retrieval.jsonl (embed_query, status ok)	Local Ollama. The README reports about 110 ms per search through the API and about 12
	rag/eval/faithfulness_labels.jsonl on branch cursor/deepeval-rag-eval	Human labels for checking a judge. No judge run against them is committed, so no judge ag

2.3 Missing data

Requested metric	Status in the repository
Context Precision (RAGAS-style, LLM-judged)	Not measured. The DeepEval branch can compute contextual relevancy with --llm, but no output is committed.
Context Recall (RAGAS-style, LLM-judged against a reference answer)	Not measured. The benchmarks have gold sections but no reference answers.
Faithfulness (LLM-judged)	Not measured. Only the deterministic "unbacked facts" proxy and 30 hand labels exist.
Answer Correctness	Not measured. There are no reference answers. Citation accuracy (§2.2) is the closest existing figure.
Any chunking comparison	None before this report. Only the production chunking was ever evaluated.
Retrieval latency per chunking strategy, with embeddings	Not measured (see §10).

5. Chunking techniques: what the literature says

	Weaknesses
	Cuts through sentences, rules and tables. A rule and its exception can land in different chunks.
	Still blind to document structure. A heading can be separated from its body.
	One embedding call per sentence. Chunk size is uncontrolled: the percentile rule can make very large chunks. Gains are inconsistent.
ross re-ingestion. A citation points to a meaningful unit.	Depends on parsing quality. Unstructured text needs a fallback. Section sizes vary.

The literature therefore expects structure-aware chunking to do well on documents with clear structure, like these policies and FAQs. It does not expect semantic chunking to justify its cost on short, well-structured documents. The experiment below tests both expectations on this corpus.

6. Methodology of the experiment

Script.services/knowledge/scripts/chunking_experiment.py. It is evaluation-only: the production index, retrieval and API code are unchanged.

Held constant, all taken from the existing code:

the documents and parsers (build_passages)search_knowledge, which includes query expansion, the named-document boost, preamble demotion, deduplication and the thresholdsthe tokenizer and BM25 (k1 = 1.5, b = 0.75)per-tenant shipped thresholds: shop_001 min_bm25 = 2.5, noon_eg 4.0, telco_001 4.0top_k = 5the benchmark questions, with the test split as the headlineRetrieval mode. Keyword-only (BM25), the service's own fallback. Hybrid

was blocked: the --embed run failed with EmbeddingUnavailable: ... [WinError 10061] No connection could be made (docs/reports/eval_runs/chunking/embed_attempt.txt). Ollama is not installed and Docker is not running. The script runs the identical experiment in hybrid mode with --embed once Ollama is available.

Strategies. All strategies chunk the same per-document text stream: the parsed sections, each written as title\ntext, joined by blank lines. So the only thing that changes is where the boundaries fall.

% overlap
k where the distance between neighbours exceeds the 95th or 80th percentile), using lexical TF-IDF vectors instead of embeddings, because embeddings were unavailable. In effect this is a Te
de the section

Gold matching. Gold labels are section citations. A chunk is matched to the gold section by character overlap of their spans in the stream.

A chunk is relevant if it holds at least 50% of a gold section, or at least 50% of the chunk is gold text.Hit@5: any relevant chunk in the top 5. For structure this is the existing recall@5.Context recall: the best share of one gold section's characters covered by the returned chunks. Gold citations are alternatives, as in the existing metric.Context precision: the RAGAS formula, Σ■ precision@k·rel■ / #relevant, using the reference-based relevance above instead of an LLM judge.Char precision: the share of returned characters that are gold text.No-answer precision and recall: the same definitions as evaluation.run_retrieval.These are deterministic, reference-based versions of the RAGAS context metrics, comparable to RAGAS's non-LLM variants. They are not LLM-judged scores.

Three settings on the test split:

(a) shipped thresholds, top 5(b) the same with an equal 1,500-character context budget: ranked chunks up to 1,500 characters, with the last one cut to fit. Without this, large chunks can "win" by returning most of a small corpus.(c) the keyword threshold re-picked on dev for each strategy, using the existing SWEEP_BM25 grid and the "most questions correct" ruleStatistics. Results are pooled over 158 answerable and 45 unanswerable test questions. Paired comparisons against structure use exact McNemar for hit@5 and an exact sign test for context recall (docs/reports/eval_runs/chunking/build_chunking_tables.py).

Harness check. The structure strategy run through the harness must reproduce evaluation.run_retrieval exactly. It does on all six tenant/split pairs. This is also covered by a unit test.

Harness check: structure through this harness against evaluation.run_retrieval

Tenant	Split	run_retrieval recall@5	harness hit@5	run_retrieval no-answer recall	harness no-answer recall	match
shop_001	dev	85.2	85.2	50.0	50.0	yes
shop_001	test	82.1	82.1	77.8	77.8	yes
noon_eg	dev	87.8	87.8	79.2	79.2	yes
noon_eg	test	90.0	90.0	62.5	62.5	yes
telco_001	dev	82.2	82.2	66.7	66.7	yes
telco_001	test	76.7	76.7	58.3	58.3	yes

These keyword-only figures are below the hybrid figures in §2.1, as expected: for example, shop_001 test is 82.1% here against 89.29% hybrid. Compare the strategies with each other within this report, not with §2.1.

7. Configurations and results

Chunk statistics (current documents)

Strategy	shop_001 chunks / median chars	noon_eg chunks / median chars	telco_001 chunks / median chars	chunk+BM25 build ms (sum)
structure	36 / 178	32 / 272	59 / 568	21.5
fixed-256	38 / 256	47 / 256	162 / 256	38.7
fixed-512	19 / 511	24 / 512	84 / 512	31.3
fixed-1024	11 / 1024	12 / 1024	44 / 1024	58.6
recursive-256	42 / 196	55 / 191	182 / 211	31.8
recursive-512	22 / 343	24 / 427	99 / 366	33.2
recursive-1024	11 / 854	13 / 876	50 / 747	42.4
semantic-lex-95	13 / 486	8 / 1002	37 / 800	170.1
semantic-lex-80	35 / 203	26 / 353	94 / 348	147.9
structure+recursive-512	36 / 200	36 / 286	102 / 345	29.8

Corpus size (current documents, characters): shop_001 7915, noon_eg 10165, telco_001 34537.

Pooled test split, top 5, shipped thresholds (158 answerable, 45 unanswerable questions)

%	Context precision	Context recall	Char precision	No-answer recall %	No-answer precision %	Avg returned chars	Hit@5 vs structure (won/lost, p)	Context
	0.643	0.810	0.219	64.4	65.9	2300	baseline	baseline
	0.544	0.535	0.230	57.8	68.4	1217	3/9, p=0.146	3/99, p=
	0.597	0.702	0.181	80.0	70.6	2396	4/7, p=0.549	4/58, p=
	0.641	0.765	0.120	82.2	61.7	4515	6/8, p=0.791	6/25, p=
	0.569	0.494	0.246	60.0	71.1	973	2/8, p=0.109	2/97, p=
	0.611	0.707	0.213	71.1	66.7	1878	2/2, p=1.000	2/62, p=
	0.629	0.791	0.150	80.0	63.2	3479	2/5, p=0.453	2/8, p=0
	0.644	0.790	0.104	84.4	58.5	5513	9/12, p=0.664	11/50, p
	0.628	0.733	0.220	64.4	65.9	2015	0/2, p=0.500	2/102, p
	0.616	0.697	0.239	60.0	64.3	1636	0/1, p=1.000	0/61, p=

Pooled test split, 1500-character budget, shipped thresholds (158 answerable, 45 unanswerable questions)

%	Context precision	Context recall	Char precision	No-answer recall %	No-answer precision %	Avg returned chars	Hit@5 vs structure (won/lost, p)	Context
	0.627	0.721	0.294	64.4	65.9	1426	baseline	baseline
	0.532	0.582	0.223	57.8	68.4	1416	8/3, p=0.227	16/85, p
	0.588	0.605	0.235	80.0	70.6	1479	8/16, p=0.152	15/59, p
	0.584	0.579	0.232	82.2	61.7	1497	5/26, p=0.000	11/55, p
	0.549	0.551	0.210	60.0	71.1	1378	12/3, p=0.035	17/85, p
	0.605	0.649	0.245	71.1	66.7	1447	9/7, p=0.804	17/56, p
	0.604	0.639	0.257	80.0	63.2	1489	1/11, p=0.006	4/25, p=
	0.505	0.509	0.210	84.4	58.5	1492	1/35, p=0.000	5/73, p=

	0.624	0.681	0.267	64.4	65.9	1435	9/6, p=0.607	16/90, p=
	0.617	0.678	0.258	60.0	64.3	1409	8/2, p=0.109	17/50, p=

Pooled test split, top 5, keyword threshold re-picked on dev per strategy (158 answerable, 45 unanswerable questions)

%	Context precision	Context recall	Char precision	No-answer recall %	No-answer precision %	Avg returned chars	Hit@5 vs structure (won/lost, p)	Context
	0.634	0.797	0.217	80.0	67.9	2352	baseline	baseline
	0.532	0.523	0.229	66.7	66.7	1234	3/10, p=0.092	3/97, p=
	0.585	0.685	0.176	68.9	68.9	2369	3/7, p=0.344	3/57, p=
	0.642	0.772	0.120	82.2	62.7	4510	7/6, p=1.000	7/23, p=
	0.557	0.484	0.253	71.1	62.7	975	1/9, p=0.021	1/95, p=
	0.613	0.710	0.210	71.1	71.1	1871	4/1, p=0.375	4/61, p=
	0.635	0.797	0.150	80.0	64.3	3473	3/3, p=1.000	3/6, p=0
	0.670	0.834	0.103	77.8	63.6	5469	10/4, p=0.180	12/45, p=
	0.618	0.714	0.216	71.1	66.7	2020	0/3, p=0.250	2/102, p=
	0.612	0.693	0.235	68.9	70.5	1659	2/1, p=1.000	2/61, p=

Per tenant. The per-tenant tables for every setting are in docs/reports/eval_runs/chunking/tables_keyword_only.md. The main per-tenant pattern, from the top-5 table there:

shop_001 (smallest corpus, 7,915 characters): six strategies reach 89.3% hit@5 against structure's 82.1%. That is 2 more questions out of 28. Five of the six return 1.6 to 3.7 times as much text. With top 5, fixed-1024 returns about half of the corpus per question. Under the equal budget, structure is 85.7% and only fixed-256 and recursive-256 reach 89.3%, and both have lower context recall (0.829 and 0.831, against 0.857).noon_eg: structure is best on hit@5 (90.0%, tied with structure+recursive-512) and on context recall (0.900). structure+recursive-512 has a marginally higher context precision (0.781 against 0.775).telco_001 (largest corpus, sections up to 1,164 characters): under the budget, small chunks raise hit@5 (recursive-256 77.8% against 66.7%), but they return fragments: context recall falls to 0.381 against 0.614.Latency (measured, keyword-only, CPU, this machine). Building the chunks and the BM25 index takes 21–170 ms in total across the three tenants. Search p50 is 0.3–1.6 ms at top 5 and 0.4–4.6 ms in budget mode, which fetches 20 results. p95 is at most 12.1 ms for every strategy. These are run-to-run noise levels: an earlier run of the same script gave per-strategy p50s that differed from the recorded ones by tenths of a millisecond. Latency does not separate the strategies in keyword mode. In hybrid mode, query latency is dominated by the query embedding (§2.2: p50 103 ms), which does not depend on chunking. Index-time embedding cost scales with the number of chunks, for example telco_001 has 59 chunks under structure and 182 under recursive-256. That cost was not measured. Semantic chunking with embeddings would add one embedding call per sentence at index time, which was also not measured.

8. Analysis and failure cases

On hit@5, nothing beats structure significantly at top 5. In setting (a) every alternative is within 3.8 points, and the differences are within noise (all $p \geq 0.109$). recursive-512 ties at 81.0%.Context recall favours structure, partly by construction. Gold labels are whole sections, so a strategy that splits a section can at best tie on that question. The strong losses for small chunks (for example fixed-256 won 3 and lost 99) mostly mean the answer clause came back in pieces. That matters for this product: a policy rule and its exception or condition ("within 14 days … unless the item is damaged") belong together. Still, read context recall as "complete clause returned", not as a neutral measure.Big chunks look good only when they are allowed to return more text.semantic-lex-95 and fixed-1024 match structure on context precision at top 5, but return 2.0–2.4 times as many characters, with char precision 0.104–0.120 against 0.219. On noon_eg, semantic-lex-95 returns about 6,961 of the 10,165 characters of the corpus per question. Under the equal budget both collapse (hit@5 54.4% and 62.7%, both $p < 0.001$ against structure).The equal budget is the

fairest test, and structure has the best context recall (0.721) and char precision (0.294) there. The only significant hit@5 gain over it is recursive-256 (12 won / 3 lost, p = 0.035). That gain comes at a large loss in context recall (0.551, 17 won / 85 lost), so the answer model would see partial rules.No-answer recall moves with chunk size and the threshold, not with chunk quality. Large chunks refuse more off-topic questions (fixed-1024 82.2% against 64.4%), but they also return nothing for more answerable questions (no-answer precision 61.7% against 65.9%). Re-picking min_bm25 on dev lifts structure itself to 80.0% no-answer recall (setting (c)). The weak no-answer recall reported in §2.1 is therefore a threshold and relevance-check problem, not a chunking problem.Semantic chunking (lexical proxy) did not pay off. At the 80th percentile it has lower context recall than structure in every setting, and its hit@5 is never significantly different ($p \geq 0.25$). At the 95th percentile the chunks are too large (median 1,002 characters on noon_eg). This agrees with Qu et al. Whether embedding-based semantic chunking would do better is untested.structure+recursive-512 brings no benefit. Splitting telco_001's longer sections lowered context recall (0.584 against 0.767 at top 5) and did not raise hit@5.Failures of the production chunking (keyword-only, test split, top 5: 30 of 158 answerable questions missed, listed in per_question_keyword_only.jsonl with strategy = structure, thresholds = shipped):

15 empty results (below the BM25 gate) and 15 mis-ranked.16 of the 30 are missed by every strategy. These are vocabulary gaps, not chunking:Arabizi words absent from data/shared/arabizi.json, for example FAQ-058-arabizi "sigel el mokalmat" and FAQ-060-arabizi "rasayel de3aeya".Confusion between similar telco recharge and billing sections: FAQ-010 in all three styles, FAQ-016 in Arabic and Arabizi.Questions the keyword retriever could not match to the target section, for example NQ012 "Can I return groceries?" and NQ018 on rejected returns.Wins for other strategies are mostly boundary luck. For example, Q04 ("Can I return underwear or swimwear?", gold return_policy@v2#s4, Arabic text): the English item words have no Arabic synonym, so only the expanded word "[redacted]" (return) matches. fixed-256 reaches the gold section at rank 5 only because one window straddles the end of §3 and the heading "[redacted]" ("products that cannot be returned"). The fix is a synonym entry, not a different chunker.Off-topic questions that share a policy word still return passages (16 of 45 under structure). Examples: NQ111 "Can I track my order on the noon app?" and NQ129 "dark mode" both match "app" / "noon". Larger chunks refuse some of these only because their BM25 scores are diluted.The production hybrid run (§2.1) shows the same two kinds of miss in its recorded output (eval-retrieval_telco_001_test.txt): Arabizi questions that return nothing or the wrong section (telco_001 Arabizi 19/30), and 8 off-topic questions that still return passages.

9. Recommendation

Keep the structure-aware chunking that ships today. Do not switch to fixed-size, recursive or semantic chunking. On this corpus:

it is never significantly worse than any alternative on hit@5it has the highest mean context recall in settings (a) and (b), the equal budget. In (c), semantic-lex-95 has a higher mean, but only by returning 2.3 times the text, and it loses to structure on more individual questions.it has the best char precision under the equal budgetit keeps citations meaningful (policy@version#section) and stable, which the agent's citation checks and the dashboard depend on.The retrieval errors found here come from vocabulary and thresholds, so the evidence points to these improvements instead:

Extend the synonym lists for the missed Arabizi and Arabic terms. Tune them on dev only.Re-check the no-answer side, the relevance check of returned passages and the thresholds, rather than chunk size.For onboarded tenants whose documents have no headings, parse_text falls back to paragraph blocks. If a structure-free fallback is ever needed, recursive-512 was the strongest such option at top 5 (81.0% hit@5, tied with structure, with no significant difference). This is a weak signal from keyword mode only.Confidence: moderate. The ranking held across three settings and three tenants. But it was measured in keyword mode only, on small, well-structured, partly synthetic corpora, and without answer-level metrics.

8. Limitations and remaining work

Blocked: hybrid retrieval (bge-m3). Ollama is not installed and the Docker daemon is not running. The production configuration is hybrid, so absolute numbers and possibly the ranking could change. To unblock: ollama pull bge-m3, then PYTHONPATH=src python scripts/chunking_experiment.py --embed. This writes results_hybrid.json, and python docs/reports/eval_runs/chunking/build_chunking_tables.py hybrid builds its tables. This run also covers embedding-based semantic chunking and index-time embedding cost.

Blocked: Faithfulness and Answer Correctness. There is no LLM API key or local model. The benchmark also has no reference answers, which Answer Correctness needs. To measure them:

Generate answers with the agent's knowledge prompt for each strategy's top 5.Score them with the DeepEval harness on cursor/deepeval-rag-eval, after merging it onto the current services/knowledge layout (it still imports team_a.*).Calibrate the judge against the 30 hand labels.Write reference answers for the test split.Metric caveats:

Context precision and recall here are reference-based proxies (character overlap with gold sections), not LLM-judged RAGAS scores.Gold sections were defined by the structure chunker, which biases context recall towards it (§8, point 2). The hit rule (at least 50% of the chunk is gold) is lenient towards small chunks.Sample size. Pooled, there are 158 answerable test questions, but per tenant shop_001 has only 28, where one question moves hit@5 by 3.6 points. telco_001 accounts for 90 of the 158, so it dominates the pooled numbers.

Not explored:

token-based sizesother overlap ratios: 15% was fixedparent/child or "small-to-big" retrievalcontextual chunk headers, for example prefixing fixed chunks with the section titlererankers13. Reproducing and verifying

```
cd services/knowledge
PYTHONPATH=src python scripts/chunking_experiment.py            # writes docs/reports/eval_runs/chunking/results_keyword_only
cd ../..
python docs/reports/eval_runs/chunking/build_chunking_tables.py # writes tables_keyword_only.md + pooled_keyword_only.json
python docs/reports/eval_runs/chunking/verify_report.py         # checks this report against those files and the existing res
cd services/knowledge && python -m pytest -q                    # includes tests/test_chunking_experiment.py
```

The retrieval scores are deterministic. A second run into a scratch folder gave identical per-question output and identical results apart from timing fields (latency and build ms). Because the chunk-statistics table embeds build ms, verify_report.py passes against the committed outputs; after a rerun, rebuild the tables and recopy that table first.

Files

Experiment: services/knowledge/scripts/chunking_experiment.pyTests: services/knowledge/tests/test_chunking_experiment.pyOutputs: docs/reports/eval_runs/chunking/ (results_keyword_only.json, per_question_keyword_only.jsonl, tables_keyword_only.md, pooled_keyword_only.json, embed_attempt.txt, build_chunking_tables.py, verify_report.py)3. New evaluation: BM25 + Groq LLM judges (2026-10-30)

Source: docs/reports/eval_runs/rag_complete/bm25_groq_summary.json, run at 2026-10-30T17:01:07+00:00.

Configuration: - Model: Groq openai/gpt-oss-120b for answer generation and LLM judging - Retrieval mode: BM25-only (keyword fallback). Embeddings unavailable: Ollama connection failed (WinError 10061). - Limitations: Hybrid retrieval not tested. LLM judge uses same model as answer generator (self-evaluation).

3.1 Retrieval results: BM25-only mode

Tenant	Split	Questions	Recall@5 %	No-answer precision %	No-answer recall %
shop_001	dev	33	41.4	18.2	33.3
shop_001	test	37	48.4	53.3	88.9
noon_eg	dev	65	54.7	38.5	41.7
noon_eg	test	64	48.0	38.5	41.7
telco_001	dev	102	57.8	16.7	41.7

telco_001	test	102	58.9	14.8	33.3

Comparison with §2.1 baseline (hybrid retrieval):

These BM25-only figures are substantially lower than the hybrid (bge-m3 + BM25) baseline reported in §2.1. For example: - shop_001 test: 48.4% (BM25-only) vs. 89.29% (hybrid) - noon_eg test: 48.0% (BM25-only) vs. 97.5% (hybrid) - telco_001 test: 58.9% (BM25-only) vs. 82.22% (hybrid)

The drop confirms that embeddings contribute significantly to retrieval performance. The hybrid vs. BM25-only comparison is blocked until Ollama/bge-m3 becomes available.

3.2 LLM-judged answer quality (test split only)

Tenant	N judged	Mean faithfulness	Mean answer correctness
shop_001	13	0.692	0.615
noon_eg	22	0.773	0.682
telco_001	42	0.452	0.310

Notes: - LLM judge evaluates only the test split, on questions where the retriever returned results. - Faithfulness: whether the answer is grounded in the retrieved passages. - Answer correctness: whether the answer addresses the user's question accurately. - telco_001 scores are notably lower, likely due to the larger corpus (34,537 characters, 59 sections) and more complex multi-step policies. BM25-only retrieval struggles more here. - Self-evaluation bias: The same model (Groq openai/gpt-oss-120b) generates and judges answers. Independent judge validation is needed.

4. Failure analysis

4.1 Retrieval failures (test split, BM25-only)

Summary by tenant:

Tenant	Total questions	Missed retrievals	Hallucination gaps (empty → answered)	No-answer errors (should be empty)	No-answer errors (should answer)
shop_001	37	15	6	7	1
noon_eg	64	16	16	10	3
telco_001	102	28	25	24	8

Failure types: - Missed retrieval: Expected citation not in top 5 results. - Hallucination gap: Retrieval returned nothing (below BM25 threshold), but the LLM generated an answer anyway (detector failure or post-retrieval generation issue). - No-answer precision error: Question has no expected answer (gold = []), but retrieval returned passages. - No-answer recall error: Question has expected answer, but retrieval returned empty.

4.2 Sample failures

ID	Question	Style	Expected	Got	Type
Q01	How many days do I have to return an item?	en	return_policy@v2#s2	shipping_policy@v1#s2, shipping_policy@v1#s6	missed_retrieval, no_answer_returned
Q03	momken araga3 el 7aga ba3d kam yom?	arabizi	return_policy@v2#s2	(empty)	missed_retrieval, hallucination_gap
Q40	mawa3eed el shoghl emta?	arabizi	faq@v1#q01	(empty)	missed_retrieval, hallucination_gap
Q57	ana mesh 3aref a5tar anhy ma2as	arabizi	faq@v1#q03	(empty)	missed_retrieval, hallucination_gap

D	Question	Style	Expected	Got	Type

Q006	Can I return a perfume after I've tried it?	en	noon_return_policy@v1#s5	s17, s3, s30, s24, s22	missed_retrieval, no_answer_returne
Q012	Can I return groceries?	en	noon_return_policy@v1#s11, s19	s21, s20, s0, s2, s17	missed_retrieval, no_answer_returne
Q034	[redacted]	ar	noon_return_policy@v1#s2	(empty)	missed_retrieval, hallucination_gap
Q042	[redacted]	ar	noon_return_policy@v1#s11, s19	(empty)	missed_retrieval, hallucination_gap

ID	Question	Style	Expected	Got	Type
FAQ-004-arabizi	el 5at by2fel law mastakhdemtosh?	arabizi	prepaid_recharge_policy@v3#s4	(empty)	missed_retrieval, hallucination_gap
FAQ-008-arabizi	a7awel rased le ra2am tany ezay?	arabizi	prepaid_recharge_policy@v3#s6	(empty)	missed_retrieval, hallucination_gap
FAQ-010 (all styles)	I recharged but balance didn't arrive	en/ar/arabizi	postpaid_billing_policy@v2#s5	wrong sections or empty	missed_retrieval
FAQ-012-arabizi	lazem adfa3 el fatora emta?	arabizi	postpaid_billing_policy@v2#s2	(empty)	missed_retrieval, hallucination_gap

4.3 Arabic and Arabizi vocabulary issues

BM25-only retrieval struggles significantly with Arabizi and informal Arabic:

Arabizi vocabulary gaps: Many Arabizi terms in questions are not present in the policy documents or the synonym expansion file (data/shared/arabizi.json). Examples:"mawa3eed el shoghl" (work hours)"sigel el mokalmat" (call log)"rasayel de3aeya" (promotional messages)"el 5at by2fel" (line gets locked)

Arabic term mismatches: Some Arabic questions use colloquial or synonym terms not found verbatim in the formal policy text:

"[redacted]" (product without return label) → policy uses different phrasing"[redacted]" (food products) vs. policy text variations

Impact: Arabizi questions have notably lower recall on telco_001 (dev: 27/30 → test: 19/30 in hybrid mode §2.1; worse in BM25-only).

Recommendation: Expand arabizi.json with terms from missed questions in the dev split, then re-evaluate.

4.4 No-answer misclassifications

Should-be-empty returned results (low no-answer precision): - telco_001 has the worst no-answer precision (14.8% on test): 10 of 12 unanswerable questions still returned passages. - Common cause: questions share policy-domain vocabulary (e.g., "app", "noon", "bundle") even when off-topic.

Should-answer returned empty (low no-answer recall): - shop_001 dev: 50% no-answer recall in §2.1 (hybrid), 33.3% in BM25-only. - Cause: relevant documents fall below the BM25 gate (min_bm25 threshold).

Observation: No-answer performance is primarily a threshold-tuning and relevance-gating problem, not a chunking problem (confirmed in §8 of the chunking experiment).

4.5 LLM judge failure notes

Self-evaluation bias: The same model judges its own outputs. Independent validation needed.Faithfulness detection limits: The judge assigns binary faithfulness scores. Nuanced grounding issues (partial support, misleading phrasing) may be missed.No reference answers: Answer correctness is judged without gold-standard reference answers, relying on the judge's interpretation of the question and retrieved context.10. Remaining work

Done:

■ LLM-based evaluation (faithfulness, answer correctness) using Groq openai/gpt-oss-120b (§3.2)■ Chunking comparison across 9 strategies in keyword-only mode (§5–§9)■ Retrieval metrics (recall@5, no-answer precision/recall) on all tenants, dev + test splitsRemaining:

■ Hybrid retrieval comparison (blocked on Ollama/bge-m3 embeddings): The production system uses hybrid retrieval (0.65·cosine + 0.35·BM25). BM25-only results in §3 are not representative of production performance. To unblock: Install Ollama, pull bge-m3, and rerun chunking_experiment.py --embed.■ Independent LLM judge validation: Current judge is the same model as the generator. Validate against the 30 hand-labeled faithfulness examples in rag/eval/faithfulness_labels.jsonl (branch cursor/deepeval-rag-eval).■ Reference answer creation: The benchmark has gold citations but no reference answer text. RAGAS-style answer correctness and context recall require reference answers.■ Vocabulary expansion: Add missed Arabizi/Arabic terms from dev split failures to data/shared/arabizi.json, then re-evaluate.■ Threshold re-tuning: Re-pick min_bm25 and no-answer gates per tenant using dev split (setting (c) in §7 shows this can lift no-answer recall to 80%).■ PDF report generation: Formatted report with charts (separate task, not blocking evaluation).11. Limitations

Embeddings unavailable: Ollama connection failed (WinError 10061). All new experiments in §3 ran in BM25-only mode, which underperforms the production hybrid retrieval by 20–50 percentage points on recall@5.Hybrid vs. dense comparison not run: Cannot compare bge-m3 embeddings, semantic chunking with embeddings, or measure embedding-time cost per chunking strategy.LLM judge self-evaluation bias: The same Groq model generates answers and judges them. Faithfulness and answer correctness scores may be inflated.No reference answers: Gold labels are citations only. Answer correctness is judged by the LLM without ground-truth answer text.Sample size per tenant: shop_001 has only 28 answerable test questions (37 total); one question moves hit@5 by 3.6 points. telco_001 (90 questions) dominates pooled results.Groq API rate limits: All questions were evaluated in this run (no skipping). Future runs with larger tenants may hit rate limits if the API key tier is unchanged.Metric caveats: Context precision and recall (§7) are reference-based proxies using character

overlap, not LLM-judged RAGAS scores.12. API costs and latency

Groq usage (§3 evaluation run):

Metric	Value
Total API calls	289
Input tokens	64,303
Output tokens	23,270
Estimated cost	$0.0563 USD

Cost per tenant (approximate): - shop_001 test: 13 judged questions → ~$0.0026 - noon_eg test: 22 judged questions → ~$0.0043 - telco_001 test: 42 judged questions → ~$0.0086

Latency: - Query embedding latency (§2.2, hybrid mode, local Ollama bge-m3): p50 103 ms, p95 142 ms - Groq inference latency: Not separately logged in this run. Typical Groq latency for models in this class is 100–500 ms per generation depending on output length. - BM25 search latency (§7, chunking experiment): p50 0.3–1.6 ms, p95 ≤ 12.1 ms (negligible compared to embedding or LLM call).

Observation: Query embedding (when available) and LLM generation dominate end-to-end latency. Retrieval algorithm (BM25 vs. hybrid) and chunking strategy have minimal impact on search time in keyword mode.

11. References